Manual coding protocol and codebook

Assessment of generated software requirements

This protocol specifies how to assess generated software requirements against a project's gold-standard requirements and source code. It defines the coding procedure, the nine variables recorded in the manual analysis workbook, and the conventions for recording and reviewing decisions.

Purpose and preparation

Scope. The analysis covers systems that have source code in the order of 8 000 - 24 000 lines of code, together with requiremet sets for the system, here referred to as the gold standard requirement set. For the system, requirements have been generated.

Materials. Requires the generated requirements, the complete gold-standard requirement set for the same project, the corresponding source-code version, and the manual analysis workbook. Record the versions of the materials used so that the same evidence can be examined again.

Read the gold standard. Read the complete requirement set before coding. Identify its typical presentation, semantic focus, functional scope and level of detail. Use the project's own gold standard as the reference throughout the analysis.

Read the source code. Before assessing individual requirements, read the project's source code to understand its main functionality and organisation. Identify the main classes, their responsibilities, and how user-facing actions connect to controllers, services and data-access code where these structures exist. Follow representative execution paths to understand which classes implement each functional area.

Continue reading the relevant classes and methods while coding each requirement. Initial familiarisation provides an overview; identifying a code class requires inspecting the implementation of the particular key concept and following relevant calls as needed.

Coding procedure

Unit of analysis. One generated requirement is one record, identified by project, generation variant and requirement ID. Secondary-concept and detail-match ratings apply separately to each generated–gold requirement pair. Many-to-many mappings between gold standard and generated requirements are permitted.

Read each generated requirement in full and identify its primary functionality or user goal. Assess the nine variables in the codebook order, which follows the manual analysis workbook. Examine the full project gold standard requirement set when mapping requirements and the relevant implementation when identifying code classes.

Assess secondary concepts and details only for mapped pairs. Assess abstraction and detail level for both matched and unmatched requirements. Evaluate recommended additions for unmatched functionality. Preserve the generated text and record the decisions separately.

Concept decomposition. Before assigning the comparison variables, separate the generated requirement into: (1) its key concept, defined by the main domain action and its primary object or intended outcome; (2) secondary concepts, consisting of specified actors/roles and additional domain actions; and (3) details, consisting of data, values, formats, constraints, conditions, validations and prescribed steps. Do not use differences assigned to one category to determine another.

For a mapped requirement, use the mapped gold requirement(s) as the primary reference when assessing functional scope. For an unmatched requirement, use the typical functional scope of the complete project gold set.

When a decision remains ambiguous after applying the rules, do not infer missing information. Record the competing interpretations and retain the item for review rather than resolving the ambiguity through an unstated assumption.

Coding criteria

The variable names and sequence below follow Sheet1 of the manual analysis workbook. Use the allowed labels consistently and keep the assessments of presentation, meaning, scope and detail separate.

1 Syntactic adherence

Definition. Adherence to the presentation and syntactic format of the project's gold-standard requirements.

Output and applicability. Yes or No for every generated requirement.

Decision rules. Record Yes when the same general structural pattern is clearly present, including relevant headings, sentence structure and style. Equivalent heading names are acceptable. Record No when the pattern is absent or substantially different. General readability alone does not establish adherence.

Example. If the gold standard uses named use cases with actors and a flow, an equivalent structured use case receives Yes; a plain prose sentence receives No. For a gold standard written as prose, prose is not itself grounds for No.

2 Semantic adherence

Definition. Whether the generated requirement reflects the same domain-level semantic focus as the project's gold-standard requirements.

Output and applicability. Yes or No for every generated requirement.

Decision rules. Record Yes when the requirement describes a domain-relevant goal, behavior, capability, rule, or constraint of the kind represented in the gold standard. Record No when it instead describes an implementation-specific or technical operation outside that scope. Technical details may be included without causing No if the main intent remains domain-relevant.

Example. “The employee views book information” receives Yes because it describes domain-relevant behavior. “The system loads the book object into a session” receives No because it describes an internal implementation operation rather than domain-level behavior.

3 Key concept code class

Definition. The highest-level code class or classes implementing the generated requirement’s primary functionality.

Output and applicability. A comma-separated list of class names; inspect the code for every generated requirement.

Decision rules. Locate candidate classes using the main action and affected entity. Read the relevant methods and follow calls to establish the implemented behavior. Select the highest-level class that implements or coordinates the key concept rather than listing every supporting dependency. List multiple classes where distinct parts of the main functionality require them. Class names or comments alone are insufficient evidence.

Example. For an authentication requirement, record Login.java when inspection establishes that it implements or coordinates authentication. Record when no implementing class can be located and distinguish this from an incomplete inspection.

4 Key concept manually matched to

Definition. The gold-standard requirement(s) that share the generated requirement’s core functionality or user goal. Examples of key concepts include login, user deletion, and password recovery.

Output and applicability. Unique gold-standard IDs, separated by commas, for every generated requirement with a match. Record no match when none applies.

Decision rules. Examine the complete gold set and identify the core functionality expressed by the generated requirement. Match every gold requirement that expresses the same functionality, even when the requirements differ in scope, abstraction, actor, or detail. These differences are assessed separately.

A broader generated requirement may therefore match several narrower gold requirements. Likewise, a requirement describing a substantive stage of a broader gold requirement may be matched when that stage directly contributes to the same functional goal. Do not match requirements based only on shared terminology, data, prerequisites, or incidental supporting steps.

Example. “Search users” does not match “Delete users” merely because searching may be a step in deletion. However, a requirement describing the deletion of a selected user can match “Delete users” even if it covers only one stage or variant of the full gold-standard requirement.

5 Secondary concept match

Definition. Coverage of the mapped gold requirement’s secondary domain concepts: its specified actors or roles and additional user or domain actions beyond the shared key concept.

Output and applicability. Yes, Partly, or No for each mapped gold ID, in the same order as the ID list. Not applicable for an unmatched requirement.

Rating	Decision rule
Yes	All specified secondary concepts are covered.
Partly	At least one secondary concept is covered and at least one is missing or different.
No	None of the specified secondary concepts are covered.

Decision rules. Treat the mapped gold requirement as fixed; do not reconsider the key-concept match or abstraction level in this step. Identify its explicitly specified actors/roles and additional domain actions beyond the shared key concept, and compare these as discrete items with the generated requirement. Equivalent wording counts, but a generic or implied actor is not automatically equivalent to a specified role. Differences in abstraction or scope do not by themselves determine the rating; they are assessed separately under Abstraction level. Technical mechanisms, formats, validations, and other implementation details are assessed under Details match.

Example. Gold standard requirement: a customer books a delivery and selects a preferred delivery time. If the shared key concept is booking a delivery, a generated requirement covering both the customer and the time selection receives Yes. One covering the customer and booking but omitting the time selection receives Partly. One describing the booking but with a different specified actor and no time selection receives No.

6 Details match

Definition. Preservation of the mapped gold requirement's specified data fields, values, formats, constraints, conditional behavior and prescribed steps.

Output and applicability. Yes, Partly or No for each mapped gold ID, in the same order as the ID list. Not applicable for an unmatched requirement.

Rating	Decision rule
Yes	All specified details are explicitly preserved.
Partly	Nearly all details are preserved, with minor omissions or differences. Defining constraints and required behavior remain intact.
No	Defining details are omitted, generalised or changed, or most specific information is absent.

Decision rules. Treat the mapped gold standard requirement and its key and secondary concepts as fixed. Assess only the specific information used to qualify or constrain that functionality. Do not infer unstated details. A generic statement such as "validate input" does not preserve a specified validation rule. Additional generated details do not compensate for missing gold details. Differences in functionality, actors, or abstraction are assessed under the corresponding variables rather than under Details match.

Examples. Gold standard requirement: “The customer enters a delivery date in YYYY-MM-DD format, and the date must be at least two days in the future.” A generated requirement preserving both the format and date constraint receives Yes. Preserving the date constraint but omitting the format receives Partly. Saying only that “the delivery date is validated” receives No.

7 Abstraction level

Definition. The breadth of the main functionality relative to the typical functional scope of the project’s gold-standard requirements.

Output and applicability. Equal, Lower or Higher for every generated requirement, whether matched or unmatched.

Rating	Decision rule
Equal	Covers a complete goal or group of related actions of comparable scope.
Lower	Covers a narrower action, supporting operation or internal subtask.
Higher	Combines several goals or functions into a broader requirement.

Decision rules. For a mapped requirement, compare the breadth of its main functionality with the mapped gold requirement(s). Record Lower when it expresses only a stage, subfunction, exception, or supporting operation of a gold-level goal; Equal when it represents approximately the same functional unit; and Higher when it combines multiple goals that are represented separately in the gold standard. For an unmatched requirement, make the same comparison against the typical functional unit in the complete gold set. Count domain functionality rather than fields, validations, technical steps, or amount of text.

Example. If a typical gold requirement groups viewing, editing and deleting user data, deletion alone is Lower. Descriptions of internal system operations would be Lower. If the project usually specifies those actions separately, deletion may be Equal. Combining several separately specified goals into broader user management may be Higher.

8 Detail Level

Definition. The amount and specificity of concrete information relative to typical requirements in the complete project gold set.

Output and applicability. Equal, Lower or Higher for every generated requirement, whether matched or unmatched.

Rating	Decision rule
Equal	Provides a comparable amount and specificity of concrete information.
Lower	Provides fewer or less specific details about data, steps, conditions or outcomes.
Higher	Provides more extensive concrete information.

Decision rules. Assess the whole requirement against the project baseline. Concrete implementation details count; text length, repeated information, generic steps and technical

vocabulary alone do not establish higher detail. Keep the amount of detail separate from whether those details match gold.

Example. Where gold typically specifies the fields used to create a user, “create a user” has Lower detail. A requirement naming comparable fields may be Equal. A requirement adding extensive concrete validation and outcome rules may be Higher.
A requirement may have Higher detail and Details match = No.

9 Recommended addition

Definition. Whether functionality absent from the gold standard warrants inclusion as a gold-standard requirement.

Output and applicability. Yes or No for an unmatched requirement. Not applicable when its key concept is already represented by a mapped gold requirement.

Decision rules. Record Yes only when the functionality has no key-concept match in the complete gold set, is supported by the source code, is semantically appropriate for the project’s requirements, has an equal abstraction level, and represents a contextually relevant, independently meaningful domain capability at a scope suitable for the gold standard. Record No when any of these conditions is not met, including when the behavior is merely an implementation or supporting substep. Differences in wording or detail alone do not rule out a useful addition.

Example. Requirement 'delete administrator' would be a recommended addition if not matched to a gold standard requirement, if it could be identified in the code, if it is of equal abstraction level, adhere semantically, and if it is contextually relevant, for example if similar requirements for 'delete customer' existed in the gold standard.

Recording and review

Record structure

Keep each decision linked to its project, variant and requirement ID. Preserve the original requirement text. The workbook’s generated-requirement sheets have a run label on row 1, column headers on row 2 and records from row 3. Gold-standard sheets have headers on row 1. Identify fields by their names because column positions vary across sheets.

Multiple mappings

Record each matched gold ID once. Provide exactly one secondary-concept rating and one detail-match rating for each ID, in matching order. For example, an ID list of “EA1, EA2, EA3” requires three ratings in each pairwise field. Each ID must exist in the gold-standard set for the same project.

Nonapplicable and unresolved values

Distinguish a negative assessment, a nonapplicable field and an unfinished decision. Use N/A for pairwise assessments of an unmatched requirement and for Recommended addition when the key concept is already mapped. An empty mapping is a completed no-match decision only when the full gold set has been examined.

Use a separate review status or note for unresolved decisions. Do not interpret an unexplained blank code-class cell as evidence that functionality is absent, or silently convert blank rating cells to No. Preserve the meaning of historical entries when preparing an analysis copy.

Evidence and consistency

Keep brief rationales for ambiguous mappings, partial matches, code-location decisions and recommended additions. Identify relevant code paths and methods where needed to make the evidence traceable. Apply the same criteria across generation variants within a project while retaining each project's own gold-standard baseline.
Check allowed labels, valid gold IDs, alignment of pairwise lists and completion of applicable fields before computing results. Where a gold requirement specifies no secondary concepts or details, record that absence explicitly rather than inventing content to compare.

Review and version control

Record the protocol version and retain a history of changed decisions, including the previous value and reason for revision. Preserve the final coding version used for reported results. Any subsequent correction should remain traceable to that version.
For an independent assessment, provide the rater with the same protocol, requirements and source-code version. Keep the original ratings hidden until independent coding is complete. Retain both sets of decisions before discussing disagreements, and document how final decisions are resolved. Describe any completed reliability assessment and its results separately.